The AI-First iOS Developer Interview Guide

---

150 AI Interview Questions & Expert Answers

Swift • SwiftUI • Apple Intelligence • Foundation Models •
MCP • RAG • AI Agents • System Design

2026 Edition

By Ruchit B. Shah

Mobile Technology Strategist | Lead iOS Engineer | Mobile AI Enthusiast

1. Which AI tools do you use daily and why?

I use ChatGPT, Claude, Cursor, and Copilot to speed up design, coding, testing, and debugging.

I use AI as a productivity accelerator, not as a replacement for engineering judgment.

AI helps with boilerplate, architecture review, test generation, and debugging.

Example: I ask AI to generate a SwiftUI ViewModel, then I review concurrency, memory, and architecture manually.

---

2. How has AI changed your iOS development workflow?

AI helps me move faster from requirement to architecture, implementation, tests, and code review.

It reduces repetitive work but senior review is still required for production quality.

AI is useful for scaffolding, refactoring, and edge-case discovery.

Example: For a new screen, I generate MVVM structure first, then refine DI, error handling, and tests.

---

3. Which LLM is your daily driver?

My daily driver is ChatGPT/Claude depending on the task complexity.

I use ChatGPT for reasoning and architecture, Claude for long code review, and Copilot/Cursor inside IDE.

Different models are useful for different development stages.

Example: I use ChatGPT for system design and Cursor for code-level changes.

---

4. Show your favorite system prompt.

“Act as a Senior iOS Architect using Swift, SwiftUI, MVVM, Clean Architecture, async/await, DI, and unit testing.”

A good prompt defines role, architecture, constraints, coding standards, and expected output.

Prompt quality directly improves AI code quality.

Example: “Do not use force unwraps, avoid Massive ViewModel, add testable services, and explain trade-offs.”

---

5. How do you prevent AI from generating poor Swift code?

I give clear architecture constraints and always review generated code manually.

I check for SOLID, thread safety, memory leaks, cancellation, testability, and readability.

AI output must pass the same quality bar as human-written code.

Example: I reject AI code if it puts networking directly inside SwiftUI View.

---

6. How do you validate AI-generated code?

I validate it using code review, unit tests, SwiftLint, Instruments, and manual testing.

AI code can compile but still be wrong architecturally or unsafe in production.

Validation requires functional, architectural, and performance checks.

Example: For async API code, I test success, failure, timeout, cancellation, and retry cases.

---

7. What types of code do you never trust AI to write blindly?

I never blindly trust AI for security, payments, authentication, encryption, and concurrency-heavy code.

These areas have high business and security risk.

AI can hallucinate unsafe implementations.

Example: I would never accept AI-generated token storage without checking Keychain and backend flow.

---

8. Have you ever rejected AI-generated code?

Yes, I reject AI code when it breaks architecture, ignores edge cases, or creates race conditions.

AI often produces working code but not necessarily production-ready code.

Senior engineers must identify hidden risk.

Example: AI once generated shared mutable state without actor isolation; I replaced it with an actor-based store.

---

9. How do you measure productivity gains from AI?

I measure saved development time, reduced boilerplate, faster tests, and fewer review iterations.

Productivity should not be measured only by lines of code.

Quality, delivery speed, and maintainability matter more.

Example: AI may reduce initial implementation time by 30%, but code still needs review and tests.

---

10. What percentage of your code is AI-assisted?

Around 30–50% can be AI-assisted, but 100% is engineer-reviewed.

AI helps with drafts, but final ownership remains with the developer.

AI-assisted does not mean AI-owned.

Example: I may use AI for ViewModel scaffolding, but I personally finalize architecture and business rules.

---

11. How do you review AI-generated pull requests?

I review AI-generated PRs more strictly for correctness, security, concurrency, and maintainability.

AI can create confident-looking but incorrect code.

AI PRs need architectural and behavioral validation.

Example: I check whether async tasks are cancelled when the ViewModel deinitializes.

---

12. How do you avoid becoming over-dependent on AI?

I use AI for acceleration, but I keep fundamentals strong and verify every critical decision.

AI should support thinking, not replace it.

Strong iOS fundamentals are still essential.

Example: I don’t ask AI to “fix memory leaks” blindly; I verify using Instruments.

---

13. How do you keep AI output aligned with coding standards?

I provide project rules, architecture patterns, naming conventions, and examples in the prompt.

AI performs better when context and constraints are explicit.

Standardized prompts help teams maintain consistency.

Example: “Use protocol-based DI, no singleton service, add unit tests, follow MVVM.”

---

14. How do you use AI without compromising code quality?

I treat AI output as a first draft, then apply engineering review and testing.

Quality comes from validation, not generation.

AI improves speed, but responsibility stays with the engineer.

Example: I generate unit tests with AI, then manually verify assertions and edge cases.

---

15. Write a prompt to generate a production-ready SwiftUI screen.

“Create a SwiftUI screen using MVVM, async/await, DI, loading/error states, and unit-testable ViewModel.”

The prompt should include architecture, state handling, dependency injection, and testing needs.

Specific prompts produce better production code.

Example: “Create ProductListView with ProductViewModel, ProductRepository protocol, mock service, and XCTest cases.”

---

16. How do you prompt AI to follow MVVM?

I explicitly define View, ViewModel, Model, Repository, and dependency boundaries.

Without constraints, AI may mix UI, networking, and business logic.

MVVM needs clear responsibility separation.

Example: “SwiftUI View should only render state and trigger ViewModel actions.”

---

17. How do you force AI to use Clean Architecture?

I mention layers: Presentation, Domain, Data, Entities, UseCases, Repositories, and DI.

Clean Architecture protects business logic from UI and framework changes.

AI needs layer boundaries in the prompt.

Example: “Do not call API service directly from ViewModel; use FetchProductsUseCase.”

---

18. How do you reduce hallucinations?

I provide real context, ask AI to state assumptions, and verify with documentation or tests.

Hallucinations reduce when prompts include constraints and known APIs.

Trust but verify.

Example: If AI suggests a non-existing Swift API, I check Apple documentation before using it.

---

19. What information do you include in your prompts?

I include role, platform, Swift version, architecture, constraints, expected output, and edge cases.

Better context leads to better code.

Prompt should behave like a mini technical specification.

Example: “iOS 18+, SwiftUI, MVVM, async/await, no third-party library, support retry and cancellation.”

---

20. How do you ask AI to generate unit tests?

I ask for XCTest/Swift Testing with mocks, success, failure, empty, and cancellation cases.

AI-generated tests must verify behavior, not implementation details.

Good tests cover realistic scenarios.

Example: “Write tests for ProductViewModel using MockProductRepository.”

21. How do you ask AI to refactor UIKit to SwiftUI?

I ask AI to preserve behavior, separate state, and migrate gradually using MVVM.

Large refactors should be incremental and low risk.

UIKit-to-SwiftUI migration needs careful state mapping.

Example: Convert one UIViewController into SwiftUI View while keeping the existing service layer.

22. Show a prompt for async/await implementation.

“Convert this callback API to async/await with error handling, cancellation, and unit tests.”

Async code must handle lifecycle, cancellation, and error propagation.

Async/await improves readability but still needs safety.

Example: “Wrap legacy completion handler using withCheckedThrowingContinuation.”

23. Show a prompt for dependency injection.

“Refactor this code using protocol-based dependency injection and constructor injection.”

DI improves testability and decoupling.

AI should avoid hardcoded services.

Example: Inject `UserRepositoryProtocol` into `UserViewModel`.

24. How do you make prompts reusable across projects?

I create reusable prompt templates for architecture, testing, review, refactoring, and debugging.

Teams can standardize AI usage with shared prompts.

Prompt libraries improve consistency.

Example: A team prompt template can enforce MVVM, DI, SwiftLint rules, and test coverage.

25. Design an AI-powered chat application.

I would use SwiftUI, MVVM, streaming API, conversation storage, token management, and secure backend proxy.

The app should support streaming, cancellation, retries, persistence, and privacy.

Chat apps need real-time UX and strong state management.

Example: `ChatView → ChatViewModel → ChatUseCase → AIRepository → Backend API`.

26. Design an AI document summarizer.

I would upload documents securely, extract text, chunk content, summarize using LLM, and cache results.

Large documents need chunking, token control, and privacy handling.

Summarization requires preprocessing and result validation.

Example: PDF text is split into chunks, summarized individually, then combined into a final summary.

27. Design an AI voice assistant.

I would combine speech-to-text, LLM processing, tool calling, and text-to-speech.

Voice assistants need low latency, interruption handling, and permission management.

Audio pipeline must be reliable.

Example: User speaks → Speech framework → LLM → action → AVSpeechSynthesizer response.

28. Design an AI image analysis app.

I would use Vision/Core ML or cloud AI depending on privacy, accuracy, and device capability.

On-device is better for privacy; cloud is better for complex reasoning.

Image AI requires compression, permission, and result confidence handling.

Example: Receipt image → OCR → LLM categorizes expense → user confirms result.

29. Design an AI coding assistant.

I would build editor context extraction, prompt construction, model response, diff preview, and safe apply flow.

Coding assistants must never directly modify code without review.

Human approval is critical.

Example: AI suggests a Swift refactor, user reviews diff, then applies changes.

30. Design an AI travel planner.

I would use user preferences, destination data, tool calling, itinerary generation, and calendar integration.

Travel planning requires real-time data and personalization.

External tools improve accuracy.

Example: User asks for a 3-day trip; AI checks weather, hotels, attractions, and budget.

31. Design an AI shopping assistant.

I would combine search, recommendations, comparison, reviews, and user preference memory.

It needs product ranking, price freshness, and personalization.

Shopping AI must avoid hallucinated product data.

Example: AI compares iPhones based on budget, camera, battery, and offers.

32. How would you architect multiple AI providers?

I would define an `AIProviderProtocol` and implement providers like OpenAI, Anthropic, Gemini, or Apple.

Provider abstraction avoids vendor lock-in.

The app should not depend directly on one SDK.

Example: `OpenAIProvider`, `ClaudeProvider`, and `AppleFoundationModelProvider` conform to the same protocol.

33. How would you switch between OpenAI, Anthropic, and Gemini?

I would use provider abstraction, remote config, and backend routing.

Switching should not require UI or domain layer changes.

Keep model selection outside presentation logic.

Example: Backend chooses provider based on cost, latency, and availability.

34. Where should AI business logic live in MVVM?

AI business logic should live in UseCase/Domain layer, not directly in View or ViewModel.

ViewModel should coordinate state, not own AI rules.

This keeps code testable and scalable.

Example: `GenerateSummaryUseCase` calls `AIRepository`, while ViewModel only exposes loading/result/error.

35. How do you integrate an LLM API into an iOS app?

I integrate through a secure backend proxy, not directly with API keys in the app.

Mobile apps can be reverse engineered, so keys must stay server-side.

Backend handles auth, rate limits, logging, and provider calls.

Example: iOS sends prompt to backend; backend calls LLM and streams response back.

36. Explain streaming responses.

Streaming means receiving partial AI output token-by-token instead of waiting for the full response.

It improves perceived performance and user experience.

Streaming is common in chat apps.

Example: The answer appears progressively like ChatGPT instead of after 10 seconds.

37. How do you display streaming tokens in SwiftUI?

I update an observable state on the MainActor as tokens arrive.

UI updates must happen safely on the main thread.

AsyncSequence works well for streaming.

Example: `for await token in stream { await MainActor.run { text += token } }`.

38. How do you cancel an ongoing AI request?

I keep a Task reference and call `cancel()` when the user stops or leaves the screen.

Cancellation prevents wasted cost, battery, and incorrect UI updates.

Always respect lifecycle.

Example: Cancel streaming in `onDisappear`.

39. How do you retry failed requests?

I use limited retries with exponential backoff for transient failures.

Retry should not happen blindly for auth, validation, or quota errors.

Retry only recoverable errors.

Example: Retry network timeout after 1s, 2s, 4s, then show error.

40. How do you support offline mode?

I cache previous conversations and allow draft prompts offline, but AI generation needs connectivity unless using on-device models.

Offline support depends on model location.

On-device models improve offline capability.

Example: User can read old summaries offline and sync new prompts later.

41. How do you cache AI responses?

I cache deterministic responses using prompt hash, user context, and expiry rules.

Caching reduces cost and latency but must consider personalization and freshness.

Not every AI response should be cached.

Example: FAQ summaries can be cached, but personal medical advice should not be reused.

42. How do you maintain conversation history?

I store messages locally and send only relevant context to the model.

Full history increases cost and token usage.

Context trimming is important.

Example: Keep recent messages plus summarized older context.

43. How do you estimate token usage?

I estimate tokens from prompt length, model limits, and response budget.

Token planning prevents truncation and cost spikes.

Long context needs summarization or chunking.

Example: Before sending a document, I chunk it and summarize each part.

44. How do you reduce API costs?

I reduce cost using caching, smaller models, prompt optimization, streaming cancellation, and backend limits.

Cost control is part of production AI architecture.

Every unnecessary token costs money.

Example: Use a smaller model for classification and a larger model only for reasoning.

45. What is Apple Intelligence?

Apple Intelligence is Apple's personal intelligence system integrated across iOS, iPadOS, and macOS.

It combines on-device models, Private Cloud Compute, and system-level user context with privacy focus.

It helps with writing, summarization, image generation, Siri, and app intelligence.

Example: An app can use Apple intelligence APIs for smarter user experiences.

46. Which Apple Intelligence features run on-device?

Lightweight and privacy-sensitive tasks can run on-device, while complex tasks may use Private Cloud Compute.

Apple balances privacy, performance, and model capability.

On-device AI avoids sending user data to cloud.

Example: Simple text generation or classification may run locally, while complex reasoning may use cloud.

47. What is Private Cloud Compute?

Private Cloud Compute lets Apple handle complex AI requests in the cloud while protecting user privacy.

Apple says personal data sent to PCC is not accessible to anyone other than the user, not even Apple.

It extends Apple's privacy model from device to cloud.

Example: A complex Siri request may be processed using PCC if on-device model is insufficient.

48. When should you use Apple Intelligence vs a cloud LLM?

Use Apple Intelligence for privacy-first, system-integrated tasks; use cloud LLMs for advanced reasoning or cross-platform needs.

Choice depends on privacy, capability, cost, latency, and platform support.

No single model fits all use cases.

Example: Use Apple on-device AI for private summarization; use cloud LLM for multi-step business workflows.

49. How do you protect user privacy?

I minimize data collection, redact sensitive data, use backend controls, encryption, and privacy-first AI providers.

AI apps must follow data minimization and secure processing.

Never send unnecessary personal data to models.

Example: Mask phone numbers and emails before sending support tickets to AI.

50. What Apple frameworks support AI features?

Core ML, Vision, Natural Language, Speech, Foundation Models, App Intents, and Siri-related frameworks.

Apple offers both traditional ML and newer generative AI capabilities.

Framework choice depends on the use case.

Example: Use Vision for OCR, Speech for voice input, and Foundation Models for text generation.

51. What limitations exist for on-device models?

On-device models offer better privacy but have limited context window, compute power, and reasoning ability.

Device RAM, Neural Engine performance, battery, and storage limit model size. Cloud models are generally stronger for complex reasoning.

Choose the model based on latency, privacy, and complexity.

Example

Text classification runs locally, while generating a detailed travel itinerary uses a cloud LLM.

52. How would you integrate Apple Intelligence into an existing app?

I would abstract AI behind a protocol so Apple Intelligence can replace or complement cloud models.

Avoid coupling business logic to a single provider.

Keep AI implementation interchangeable.

Example

AIServiceProtocol

├── AppleAIService

├── OpenAIService

└── ClaudeService

53. Have you used Apple's Foundation Models?

Yes. I understand how Foundation Models provide on-device generative AI with privacy-first architecture.

They're ideal for summarization, rewriting, and intelligence features that don't require external cloud models.

Foundation Models reduce latency and improve privacy.

Example

Generate email summaries without sending content outside the device.

54. What are Prompt Sessions?

Prompt Sessions manage the lifecycle and context of interactions with Foundation Models.

They preserve conversation state and optimize model execution.

Think of them as AI conversation contexts.

Example

Chat history is maintained inside a Prompt Session.

55. What is Structured Output?

Structured Output returns JSON or predefined schema instead of free-form text.

It reduces parsing errors and improves reliability.

Ideal for production applications.

Example

Instead of:

“John is 28.”

Return:

{

"name":"John",

"age":28

}

56. What is Tool Calling?

Tool Calling allows an AI model to invoke external functions or services.

The model decides when additional information is needed.

The LLM becomes an orchestrator.

Example

User:

“Book me a hotel.”

AI calls:

searchHotels()

bookHotel()

57. What is Function Calling?

Function Calling lets AI return structured arguments for predefined functions.

Unlike plain text, the model generates valid parameters.

Safer than parsing natural language.

Example

AI returns

{

"city":"Mumbai",

"date":"Tomorrow"

}

Then Swift executes:

searchWeather(city,date)

58. What is Response Streaming?

Streaming sends partial responses as they’re generated.

It improves perceived performance.

Users don’t wait for complete generation.

Example

Instead of waiting 15 seconds,

The answer appears:

Hello…

Hello Ruchit…

Hello Ruchit, here’s…

59. What is Prompt Chaining?

Prompt Chaining breaks a complex task into multiple smaller prompts.

Each prompt solves one step.

Improves accuracy.

Example

Step 1

Extract entities

↓

Step 2

Summarize

↓

Step 3

Translate

↓

Step 4

Generate email

60. When should you use local vs remote models?

Local models prioritize privacy and latency, while remote models provide stronger reasoning.

Select based on task complexity.

Hybrid AI architecture is becoming common.

Example

Grammar correction → Local

Medical diagnosis assistant → Cloud

61. What is MCP?

MCP is an open protocol that allows AI models to communicate with external tools and data sources.

It standardizes how AI accesses resources.

Think of MCP as “USB-C for AI.”

Example

AI accesses

Calendar

GitHub

Database

Slack

through MCP.

62. Why is MCP becoming popular?

It removes custom integrations between every model and every tool.

One protocol works with many providers.

Simplifies AI ecosystem.

Example

One MCP server works for multiple LLMs.

---

63. How is MCP different from REST?

REST exposes APIs, while MCP exposes AI-friendly tools with context.

MCP includes tool discovery and structured interaction.

REST is API-centric.

MCP is AI-centric.

Example

REST

GET /weather

MCP

Tool:

GetWeather()

64. How does an iOS app connect to an MCP server?

The app communicates through an MCP client over supported transports like HTTP or streams, usually via a backend.

The backend often manages authentication and tool access.

Never expose sensitive credentials directly.

Example

Swift App

↓

Backend

↓

MCP Server

↓

Tools

65. How do tools work in MCP?

The server exposes capabilities, and the model chooses when to invoke them.

The AI decides based on user intent.

Tool execution is automatic.

Example

User

“Send email”

↓

LLM

↓

Email Tool

66. What is security concerns exist with MCP?

Authentication, authorization, prompt injection, and tool permissions.

Never allow unrestricted tool execution.

Least privilege principle.

Example

Calendar tool cannot delete events unless permitted.

67. How would you build an MCP client in Swift?

I’d create protocol-based transport, authentication, tool discovery, and execution layers.

Use Clean Architecture to isolate networking.

Keep MCP provider-independent.

Example

Transport

↓

MCPRepository

↓

ViewModel

68. How do you manage authentication with MCP?

OAuth, JWT, API gateway, and short-lived access tokens.

Authentication should never happen in UI.

Backend owns credentials.

Example

App

↓

Backend

↓

OAuth

↓

MCP

69. What is an AI Agent?

An AI Agent can reason, plan, use tools, and complete tasks autonomously.

Unlike chatbots, agents take actions.

They combine LLM + Memory + Tools.

Example

AI books flights automatically.

70. Difference between chatbot and AI agent?

A chatbot answers questions; an AI agent completes tasks.

Agents have planning capabilities.

Chatbot

↓

Respond

Agent

↓

Act

71. Difference between AI assistant and AI agent?

Assistants respond to users, while agents proactively execute workflows.

Agents can make decisions across multiple steps.

Assistant = Conversation

Agent = Automation

Example

Assistant

“Here are flights.”

Agent

“I booked one.”

72. Explain multi-agent systems?

Multiple specialized AI agents collaborate to solve complex problems.

Each agent has a specific responsibility.

Similar to microservices.

Example

Planner

↓

Research

↓

Booking

↓

Payment

73. How would you build an autonomous travel planner?

Planner agent + booking agent + payment agent + notification agent.

Each performs one responsibility.

Task delegation improves reliability.

Example

User

↓

Planner

↓

Hotel

↓

Flight

↓

Calendar

74. How would you build an AI coding assistant?

Editor context, LLM, code analysis, diff preview, and approval workflow.

Never modify production code automatically.

Human review remains mandatory.

Example

Generate Diff

↓

Review

↓

Apply

75. How do agents use memory?

Agents store previous interactions and relevant facts to improve future decisions.

Memory can be short-term or long-term.

Context improves personalization.

Example

Remember preferred programming language.

76. How do agents call external tools?

Through Tool Calling or MCP.

The LLM selects appropriate tools based on intent.

Tools extend model capabilities.

Example

Weather

Calendar

Maps

Payments

77. Where should API keys be stored?

On the backend, never inside the iOS application.

Apps can be reverse engineered.

Use backend token exchange.

Example

App

↓

Backend

↓

OpenAI

78. What is Prompt Injection?

Prompt Injection attempts to manipulate AI into ignoring instructions.

Treat prompts like untrusted input.

It’s similar to SQL Injection for LLMs.

Example

Ignore previous instructions…

79. What is Jailbreak Prompting?

Attempts to bypass model safety restrictions.

Applications should validate model outputs and constrain prompts.

Never trust user prompts blindly.

Example

“Act as if all rules don’t exist.”

80. How do you protect sensitive user data?

Encrypt data, minimize collection, redact PII, and process locally when possible.

Privacy should be built into the architecture.

Only send what’s necessary.

Example

Remove emails before sending text for summarization.

81. How do you prevent data leakage?

Mask confidential information and use secure backend processing.

Never expose internal business data unnecessarily.

Least data principle.

Example

Replace customer IDs before sending prompts.

82. How do you secure AI conversations?

Encrypt storage, use authenticated APIs, and avoid storing sensitive prompts unnecessarily.

Conversation history is user data.

Protect both transit and storage.

Example

AES encrypted local database.

83. How do you handle authentication?

OAuth, JWT refresh, Keychain, and backend authorization.

Authentication should remain independent of AI.

Reuse existing identity systems.

Example

Login

↓

JWT

↓

Backend

↓

AI

84. How do you implement rate limiting?

Backend quotas, user-level limits, exponential backoff, and retry headers.

Protect infrastructure and reduce abuse.

Rate limiting improves stability.

Example

20 prompts/minute.

85. How do you test AI responses?

Golden datasets, deterministic prompts, and human evaluation.

AI testing focuses on quality, not exact wording.

Evaluate behavior.

Example

Expected summary contains key facts.

86. How do you mock LLM APIs?

Use protocol-based repositories with mock responses.

Unit tests shouldn’t call production AI services.

Dependency injection simplifies testing.

Example

MockAIRepository.

87. How do you test streaming?

Simulate token-by-token responses.

Verify UI updates incrementally.

Streaming is asynchronous.

Example

Assert text grows over time.

88. How do you test prompt changes?

Compare outputs using the same benchmark dataset.

Regression testing detects quality degradation.

Treat prompts like code.

Example

Version prompts.

89. What are Golden Datasets?

A fixed collection of prompts with expected outputs for evaluation.

Useful for regression testing.

AI quality benchmark.

Example

100 customer support prompts.

90. How do you test AI latency?

Measure first token, total response time, and completion rate.

User experience depends on perceived speed.

Streaming improves responsiveness.

Example

TTFT = Time To First Token.

91. How do you snapshot AI-generated UI?

Generate deterministic mock responses and run snapshot tests.

Never rely on live AI during UI tests.

Mock everything.

Example

Fixed markdown response.

92. How do you evaluate model quality?

Accuracy, latency, cost, hallucination rate, and user satisfaction.

Quality isn’t only intelligence.

Balance multiple metrics.

Example

Model A

95% accuracy

2 sec

Model B

98%

12 sec

93. How do you stream responses using AsyncSequence?

Consume streamed tokens with `for await` and update UI on the MainActor.

AsyncSequence integrates naturally with Swift Concurrency.

Ideal for streaming AI.

Example

Append each token to the displayed message as it arrives.

94. How do you cancel streaming?

Keep a Task reference and cancel it when appropriate.

Always check for cancellation during streaming.

Avoid wasted resources.

Example

User presses Stop.

95. How do you debounce prompts?

Delay requests briefly so rapid typing doesn’t trigger multiple API calls.

Debouncing reduces cost and unnecessary requests.

Useful for AI search.

Example

300 ms debounce before querying.

96. How do you avoid duplicate requests?

Track request IDs, cancel previous tasks, and deduplicate identical prompts.

Prevents wasted compute and inconsistent UI.

One request per intent.

Example

Ignore repeated search text.

97. How do actors help AI apps?

Actors protect shared mutable state from race conditions.

Perfect for conversation history and token caches.

Thread-safe by design.

Example

ConversationActor stores all messages.

98. How do you protect shared conversation state?

Use actors or other synchronization mechanisms to serialize access.

Multiple async tasks shouldn't mutate the same state simultaneously.

Prevent data corruption.

Example

Only the actor updates chat history.

99. How do you avoid race conditions?

Use Swift Concurrency, actors, immutable data where possible, and proper task coordination.

Avoid unsynchronized shared mutable state.

Thread safety is critical.

Example

Use an actor instead of multiple concurrent array writes.

100. How do you manage multiple simultaneous AI requests?

Prioritize requests, cancel stale ones, isolate state, and coordinate with structured concurrency.

Different requests should not interfere with each other or the UI.

Use `TaskGroup`, actors, and cancellation to keep the app responsive.

Example

A chat response, document summary, and image analysis can run concurrently, while each maintains its own state and reports progress independently.

101. Build a ChatGPT interface in SwiftUI

I would use SwiftUI, MVVM, streaming responses, Markdown rendering, and a conversation repository.

The UI should remain lightweight while business logic stays in the ViewModel and Domain layers.

A chat interface is primarily a state-management problem.

Example

```
ChatView → ChatViewModel → AIRepository → Backend → LLM
```

102. How do you implement streaming text?

Consume streamed tokens with `AsyncSequence` and update the UI on the `MainActor`.

Streaming improves perceived responsiveness by displaying partial output immediately.

Each incoming token is appended to the current message.

Example

Display `"Hello..."` before the full response is generated.

103. How do you show a typing animation?

Use a typing indicator while awaiting the first streamed token.

This reassures users that the request is being processed.

The indicator disappears once streaming begins.

Example

Show three animated dots before the AI response appears.

104. How do you render Markdown?

Use SwiftUI's `AttributedString` or a Markdown parser to render formatted responses.

LLMs commonly return headings, lists, tables, and code blocks.

Support rich text instead of plain strings.

Example

Render `**bold**`, lists, links, and tables correctly.

105. How do you render code blocks?

Use a monospaced font with syntax highlighting and a copy button.

Developers expect readable code formatting.

Separate code from regular text visually.

Example

Swift code appears in a highlighted container with a "Copy" action.

Persist conversations locally and synchronize with the backend when appropriate.

Users expect continuity across sessions.

History should be searchable and deletable.

Example

Store prompts in SwiftData or Core Data with timestamps.

107. How do you add speech-to-text?

Use Apple's Speech framework with microphone permissions.

Convert audio into text before sending it to the AI model.

Voice becomes another input method.

Example

108. How do you add text-to-speech?

Use `AVSpeechSynthesizer` to read AI responses aloud.

Improve accessibility and hands-free usage.

Voice output complements text output.

Example

The AI reads the generated summary.

109. How do you add image generation?

Send image prompts to an image-generation model and display the returned image URL or asset.

Keep image generation behind a backend to protect API credentials.

Separate text and image providers if needed.

Example

Prompt: “Generate a futuristic iPhone wallpaper.”

110. How do you add file upload?

Use `FileImporter`, upload securely, extract content, and pass it to the AI.

Support PDFs, images, and documents with validation.

Preprocess files before inference.

Example

Upload a PDF and ask the AI to summarize it.

111. What is RAG?

RAG combines retrieved documents with an LLM prompt to generate grounded answers.

It improves accuracy without retraining the model.

Retrieve first, generate second.

Example

Search company policies, then answer using those documents.

112. Why use RAG instead of fine-tuning?

RAG keeps information current without retraining the model.

Fine-tuning changes model behavior, while RAG supplies fresh knowledge at runtime.

RAG is usually faster and cheaper to maintain.

Example

Update a product catalog by changing documents instead of retraining.

113. Explain embeddings?

Embeddings convert text into vectors representing semantic meaning.

Similar concepts are placed closer together in vector space.

Embeddings power semantic search.

Example

“Automobile” and “car” have similar embeddings.

114. Explain vector search?

Vector search finds semantically similar content instead of exact keyword matches.

It compares embeddings rather than raw text.

Great for AI-powered search.

Example

Searching “refund” can also retrieve documents about “returns.”

115. How would you build document search?

Extract text, generate embeddings, store vectors, retrieve relevant chunks, then use RAG.

Chunking and ranking improve retrieval quality.

The search pipeline feeds the LLM.

Example

Search an employee handbook and answer HR questions.

116. Which vector database would you choose?

I choose based on scale, latency, cost, and operational needs.

Managed services reduce operational overhead, while self-hosted options provide more control.

There is no universal best choice.

Example

A startup might use a managed vector database, while an enterprise may self-host for compliance.

117. How do you keep documents updated?

Re-index documents whenever content changes.

Maintain versioning and incremental updates to avoid rebuilding everything.

Fresh data improves answer quality.

Example

Only re-embed modified documents overnight.

118. How do you rank retrieved results?

Combine vector similarity with metadata and business rules.

Recency, relevance, permissions, and source quality all matter.

Retrieval is more than nearest-neighbor search.

Example

Prioritize the latest company policy over older versions.

119. How do you reduce response latency?

Use streaming, caching, smaller models, and optimized prompts.

Measure Time to First Token (TTFT) and total completion time separately.

Perceived speed is as important as actual speed.

Example

Show streamed output after one second instead of waiting eight seconds.

120. How do you reduce token usage?

Trim unnecessary context, summarize history, and use structured prompts.

Every unnecessary token increases latency and cost.

Efficient prompts save money.

Example

Replace a long conversation with a concise summary.

121. How do you optimize prompts?

Be specific, provide constraints, and request structured output.

Clear prompts reduce hallucinations and retries.

Treat prompts like code.

Example

Specify output as JSON with required fields.

122. How do you cache responses?

Cache deterministic results with expiration policies.

Avoid caching highly personalized or rapidly changing responses.

Caching reduces cost and latency.

Example

Cache a “What is MVVM?” explanation for one day.

123. How do you stream large outputs?

Display chunks progressively while managing memory efficiently.

Never wait for the full response before updating the UI.

Streaming keeps the interface responsive.

Example

Render a long report section by section.

124. How do you paginate long conversations?

Load recent messages first and fetch older messages on demand.

This reduces memory usage and improves scrolling performance.

Use lazy loading.

Example

Load the latest 50 messages initially.

125. How do you manage memory for long chats?

Summarize old conversations and keep only the active context.

Large histories increase token usage and device memory.

Compress context intelligently.

Example

Replace 500 messages with a conversation summary.

126. How do you monitor AI performance?

Track latency, cost, accuracy, failures, and user satisfaction.

Monitoring helps optimize both engineering and business outcomes.

Define measurable KPIs.

Example

Monitor Time to First Token, completion rate, and token cost per request.

127. How would you introduce AI into an engineering team?

Start with low-risk use cases, establish guidelines, and measure outcomes.

AI adoption should improve productivity without reducing quality.

Roll out incrementally.

Example

Begin with documentation, unit tests, and code reviews before production logic.

128. What AI coding standards would you define?

Require code review, testing, security validation, and architectural compliance.

AI-generated code should meet the same standards as human-written code.

Quality rules apply equally.

Example

No AI-generated code merges without peer review and automated tests.

129. How do you review AI-generated code?

Evaluate correctness, architecture, concurrency, security, performance, and maintainability.

Don’t focus only on whether the code compiles.

Review intent and implementation.

Example

Verify proper actor isolation and cancellation handling.

130. How do you train junior engineers to use AI responsibly?

Teach them to verify AI output, understand fundamentals, and explain every change.

AI should accelerate learning, not replace it.

Understanding comes before automation.

Example

Ask them to justify why AI-generated code is correct.

131. How do you measure AI ROI?

Track delivery speed, defect rate, review effort, and developer satisfaction.

ROI includes quality and maintainability, not just speed.

Use objective metrics.

Example

Measure average feature completion time before and after AI adoption.

132. When should developers avoid AI?

Avoid AI for critical security decisions, legal interpretations, and unverified production changes.

High-risk areas require expert judgment.

Human accountability remains essential.

Example

Never deploy AI-generated authentication logic without thorough review.

133. How do you ensure AI-generated code is maintainable?

Refactor it to match project architecture, coding standards, and documentation.

Generated code should look like the rest of the codebase.

Consistency improves long-term maintenance.

Example

Rename variables, split responsibilities, and add tests before merging.

---

134. How would you migrate a legacy UIKit app with AI assistance?

Use AI to automate repetitive refactoring while keeping architectural decisions manual.

Migrate incrementally to reduce risk.

Modernize safely.

Example

Convert one screen at a time from UIKit to SwiftUI.

135. Design ChatGPT for iOS.

Build a modular architecture with SwiftUI, MVVM, streaming, caching, secure backend, and conversation persistence.

Focus on scalability, responsiveness, and privacy.

Separate UI, domain, and AI providers.

Example

Chat UI → ViewModel → AI Service → Backend → Model.

136. Design Apple Notes AI.

Use local indexing, semantic search, summarization, and optional cloud reasoning.

Keep personal data on-device whenever possible.

Privacy-first architecture.

Example

Summarize meeting notes directly on the device.

137. Design Gmail Smart Reply.

Analyze email context, generate suggestions, and let the user approve them.

Never send replies automatically.

AI assists but does not replace user intent.

Example

Offer three reply suggestions after reading an email.

138. Design an AI Meeting Assistant.

Record audio, transcribe, summarize, extract action items, and sync with calendars.

Support editing before sharing results.

Combine multiple AI capabilities.

Example

Generate meeting minutes with assigned tasks.

139. Design an AI Medical Assistant.

Provide educational guidance with clear disclaimers and escalate critical cases to healthcare professionals.

Never present the AI as a replacement for medical advice.

Safety and compliance are essential.

Example

Summarize symptoms but advise emergency care for red flags.

140. Design an AI Coding IDE.

Provide code completion, refactoring, testing, debugging, and review assistance.

Keep developers in control of changes.

AI accelerates, developers approve.

Example

Suggest optimized Swift code with a diff preview.

141. Design an AI Resume Builder.

Collect user experience, optimize wording, tailor for job descriptions, and export multiple formats.

Maintain factual accuracy while improving presentation.

Personalization is key.

Example

Generate different resumes for iOS Lead and Staff Engineer roles.

142. Design an AI Document Scanner.

Capture images, perform OCR, classify content, summarize, and store securely.

Use on-device OCR when possible.

Combine Vision and AI.

Example

Scan an invoice and extract payment details.

---

143. Design an AI Voice Translator.

Speech-to-text → Translation → Text-to-speech.

Optimize latency for near real-time conversations.

Support multiple languages.

Example

Translate English speech into Japanese audio.

---

144. Design an AI Expense Manager.

Extract receipts, categorize expenses, detect anomalies, and generate reports.

Combine OCR with financial classification.

Automation improves accuracy.

Example

Scan a restaurant bill and categorize it as “Business Meals.”

---

145. Design an AI Interview Coach.

Generate interview questions, evaluate answers, provide feedback, and track progress.

Include scoring and personalized recommendations.

Practice with measurable improvement.

Example

Analyze answers for Swift Concurrency interviews.

---

146. Design an AI Photo Organizer.

Use image embeddings, clustering, and semantic search.

Support privacy with on-device processing where possible.

Organization without manual tagging.

Example

Search “beach vacation” without creating albums.

---

147. Design an AI Travel Planner.

An AI travel planner integrates user preferences, real-time data, and multi-service orchestration to generate personalized itineraries.

I would architect this by using an LLM as an orchestrator that calls tools for weather, maps, hotel booking, and flight search, ensuring state consistency and user privacy.

Create a five-day trip to Tokyo including hotel bookings based on budget and daily weather-optimized activities.

148. Design an AI Food Recommendation App.

Analyze preferences, dietary restrictions, location, and reviews to generate recommendations.

Blend personalization with live restaurant data.

Recommendations should be explainable.

Example

Suggest high-protein vegetarian meals nearby.

149. Design an AI Fitness Coach.

Track workouts, analyze progress, adapt plans, and provide motivational feedback.

Personalization should evolve based on performance.

AI acts as a training assistant.

Example

Increase workout intensity after consistent progress.

150. Design an AI Banking Assistant.

Use secure authentication, fraud detection, spending insights, budgeting, and human escalation for sensitive actions.

Security, compliance, and explainability are more important than AI sophistication in financial applications.

AI should assist users while keeping critical decisions and approvals under strong security controls.

Example

The assistant explains unusual spending patterns, recommends savings opportunities, and helps users navigate banking services, while requiring strong authentication before any transaction.